ENTREGA DO PROJETO DA DISCIPLINA
Desenvolvimento de Sistemas Backend
Fase 2

Nome do estudante: Tiago Francisco Strey

ORIENTAÇÕES PARA EXECUTAR O SISTEMA

1. **Tecnologia utilizadas:**
   - *Node.js* v22.21.0
   - *NestJS* v10
   - *TypeORM* com *SQLite*
   - *Docker* e *Docker Compose*
   - *RabbitMQ* v3 (via *Docker*)

2. **Pré-requisitos:**
   - *Node.js* v18 ou superior instalado
   - *Docker Desktop* instalado e em execução
   - *Postman* instalado

3. **Estrutura o Projeto:**

```
tiago_strey-desenvol-sistemas-backend-fase-2/
    api-gateway/            # Ponto de entrada único (porta 3000)
    servico-gestao/         # Serviço principal (porta 3001)
    servico-faturamento/    # Microsserviço de pagamentos (porta 3002)
    servico-planos-ativos/  # Microsserviço de planos ativos (porta 3003)
    docker-compose.yml      # Orquestração dos serviços
```

4. **Instalação de Dependências:**
   Execute o comando “npm install” dentro de cada pasta (api-gateway/, servico-gestao/, servico-faturamento/ e servico-planos-ativos/):

5. **Executando o sistema:**
   Execute o comando “docker-compose up --build” na pasta raiz do projeto.

6. **Verificando o RabbitMQ:**
   Acesse o painel de administração em: http://localhost:15672 .
   - Usuário: “guest”
   - Senha: “guest”

7. Testando com o Postman:

No Postman, importe o arquivo:
tiago_strey_Desenvolvimento_de_Sistemas_backend_Fase-2.postman_collection.json

Todos os endpoints apontam para http://localhost:3000 (API Gateway).

Testes cadastrados:

1. GET /gestao/clientes — Listar todos os clientes cadastrados
2. GET /gestao/planos — Listar todos os planos cadastrados
3. POST /gestao/assinaturas — Criar uma assinatura
4. PATCH /gestao/planos/:idPlano — Atualizar o custo mensal do plano
5. GET /gestao/assinaturas/(tipo) — Retorna a lista com todas as assinaturas de um determinado tipo (TODOS/ATIVOS/CANCELADOS)
6. GET /gestao/assinaturascliente/:codcli — Retorna a lista das assinaturas do cliente informado
7. GET /gestao/assinaturasplano/:codplano — Retorna a lista de assinaturas de um plano
8. POST /registrarpagamento — Solicita o registro de um pagamento
9. GET /planosativos/:codass — Retorna se plano questionado permanece ativo

8. Portas utilizadas:

SERVIÇO	PORTA
API Gateway	3000
ServicoGestao	3001
ServicoFaturamento	3002
ServicoPlanosAtivos	3003
RabbitMQ	5672
RabbitMQ Admin	15672

9. Conclusão:

O desenvolvimento da Fase 2 consistiu na implementação de dois microsserviços adicionais e um *API Gateway*, integrados ao *ServicoGestao* da Fase 1. Os principais desafios foram a configuração da comunicação assíncrona, via *RabbitMQ*, e o gerenciamento de versões do *NestJS* entre os serviços. A solução foi padronizar todos os serviços na versão 10 do *NestJS* e utilizar variáveis de ambiente para configurar as URLs do *RabbitMQ*, permitindo que o sistema funcione tanto localmente quanto via Docker.